

To implement the MUI `Slide` component with a specific container, you use the `useRef` hook in React to create a reference to a container element (like a `Box` component) and pass that reference to the `container` prop of the `Slide` component. It's often necessary to set `overflow: hidden` on the container to conceal the sliding element when it's outside the container's bounds. [Stack Overflow +2](#)

Example Implementation

This example demonstrates a pink box sliding in and out of a gray container.

jsx



```
import React, { useState, useRef } from 'react';
import { Box, Slide, FormControlLabel, Switch } from '@mui/material';

function SlideContainerExample() {
  const [checked, setChecked] = useState(false);
  // 1. Create a ref to hold the reference to the container DOM node.
  const containerRef = useRef(null);

  const handleChange = () => {
    setChecked((prev) => !prev);
  };

  return (
    <Box sx={{ width: '100%' }}>
      {/* A switch to control the 'in' prop of the Slide component */}
      <FormControlLabel
        control={<Switch checked={checked} onChange={handleChange} />}
        label="Show from target"
      />

      {/*
        2. The container Box element with a ref attached.
        Set position: relative here (or via sx) if needed for precise positioning of abso
        overflow: hidden hides the sliding item when it's outside this Box's boundaries.
      */}
      <Box
        ref={containerRef}
        sx={{
          marginLeft: 0,
          backgroundColor: 'lightgray',
          width: 400,
          height: 200,
          overflow: 'hidden',
          p: 2, // padding for visual appeal
        }}
      />
    </Box>
  );
}
```

```
    }}  
  >  
  { /*  
    3. The Slide component, passing the container ref to the 'container' prop.  
    We use containerRef.current to pass the actual DOM node.  
  */}  
  <Slide direction="right" in={checked} container={containerRef.current}>  
    { /* The child element that will be animated */}  
    <Box sx={{ width: 50, height: 50, bgcolor: 'pink' }} />  
  </Slide>  
</Box>  
</Box>  
);  
}  
  
export default SlideContainerExample;
```

This approach, demonstrated in the [MUI documentation](#) and on [Stack Overflow](#), ensures the `Slide` transition calculates its start and end positions relative to the specified `Box` container.